

Multipel inkludering

En C++-kompilator är i princip ett ganska dumt program. Den kan endast använda definitioner som finns "tidigare" under kompileringen, så t.ex. funktioner måste vara definierade antingen helt eller via prototyper innan de används. Kompilatorn klarar inte heller av att en symbol (t.ex. funktion) är definierad två gånger, även om de har samma definition. Egentligen kunde kompilatorn vägra att kompilera nedanstående program eftersom definitionerna från `iostream` är inkluderade två gånger:

```
#include <iostream>
#include <iostream>

int main () {
    cout << "Hello world!" << endl;
}
```

Så är dock inte fallet. Det vore enormt svårt att planera hur filer skall inkludera varandra om samma symbol inte får förekomma två gånger i samma fil. Det finns ett "preprocessor-trick" man kan använda som löser problemet och gör att vi inte behöver bry oss om problematiken överhuvudtaget. I t.ex. filen `iostream` finns följande definitioner (eller liknande):

```
// This file is part of the GNU ANSI C++ Library.

#ifndef __IOSTREAM__
#define __IOSTREAM__
// diverse definitioner
#endif
```

Här har vi två nya preprocessor-nyckelord, nämligen `#ifndef` och `#endif`. Om man studerar de preprocessor-direktiv som förekommer i filen ovan kan man komma till slutsatsen att det är frågan om en rudimentär `if`-sats, men som används av preprocessorn. Det första ordet `#ifndef` skall läsas *if not defined*. Den kontrollerar om symbolen som följer efter är definierad, i detta fall då `__IOSTREAM__`. Om så inte är fallet fortsätts "evalueringen" in i `#ifndef`-satsen och direkt på nästa rad definierar symbolen `__IOSTREAM__`. Efter detta följer en hel del definitioner som inte får förekomma två gånger. Sist kommer en `#endif` som ju avslutar `if`-satsen. Om samma fil nu läses in en gång till under kompileringsskedet kommer `#ifndef` att vara falsk, eftersom symbolen `__IOSTREAM__` redan är definierad, och definitionerna innanför `if`-satsen omdefinieras inte. Detta system används av så gott som alla header-filer i C++. Systemet skyddar inte mot att *filen* läses multipla gånger, endast emot att symboler definieras multipla gånger.

Normalt definierar man en symbol som har anknytning till header-filens namn. Varje headerfil bör ha en symbol som är unik. Om man har samma symbol i flera filer kommer senare filer som använder samma symbol inte att läsas in korrekt. Om man således har t.ex. en fil som heter `Definitioner.h` bör man sätta någon form av inkluderingsskdd i denna, t.ex. på detta sätt:

```
#ifndef DEFINITIONER_H
#define DEFINITIONER_H

// diverse definitioner

#endif
```

Ingen annan fil bör använda samma symbol `DEFINITIONER_H`.

Notera att `DEFINITIONER_H` inte ges något värde, utan endast status som definierad. Det finns även ett preprocessor-direktiv som kollar om en symbol redan är definierad som heter `#ifdef`, och läses *if defined*. Det finns även en `else`-del som kan användas om man vill kunna göra t.ex. två olika definitioner på av något värde. Den skrivs `#else`. Ett användningsområde för `#ifdef` är för att skriva ut debuggnings-text på skärmen. T.ex. följande är en ganska normal konstruktion:

```
int main () {
    float X, Y, Z;

    // beräkna koordinater för X, Y och Z via någon formel
    X = ...
    Y = ...
    Z = ...

#ifdef DEBUG
    cout << "X=" << X << ", " << "Y=" << Y << ", " << "Z=" << Z << endl;
#endif

    ...
}
```

Ovan används `#ifdef` för att kontrollera om en symbol `DEBUG` är definierad. Om den är definierad är programmet under debuggning och värdena för `X`, `Y` och `Z` skrivs ut på skärmen. I den slutgiltiga versionen av programmet skall ingen dylik text skrivas ut. Man kan kontrollera huruvida debuggning skall skrivas ut i detta fall genom att placera en `#define DEBUG` eller `#undef DEBUG` i någon gemensam headerfil.

[Förutgående](#)
Makron

[Hem](#)
[Upp](#)

[Nästa](#)
Externa bibliotek